Network Management in the OS

Day 3: Network and OS Foundations for Safe Robotic Actuation

Mentor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

Table of content

1. Sockets and File Descriptors
2. Blocking I/O and Where It Hurts
3. Multiplexing: Waiting on Many fds
4. Bus, Not Socket
5. Project Hands-on Preview: Build the Server

Two Days In, Looking Down

The view from Days 1 and 2.
A frame format with CRC, an MQTT bridge, per-QoS round-trip numbers. The bus felt magic; today we open the box.

Under the broker.
Mosquitto, your bridge and every laptop on the lab LAN share one mechanism: the BSD socket. It is a file the kernel pretends is the network.

Today's question.
When several talkers reach for one arm through that file abstraction, what does the OS do, where does latency hide, and how does *one slow client* bring everyone else to a halt?

1 Sockets and File Descriptors

You Have Met These Calls Before

Kurose Chapter 2, the socket programming section.
`socket()`, `bind()`, `listen()`, `accept()`, `connect()`: the calls a TCP or UDP application uses, taught from the application layer looking down.

What we add today.
The view from the kernel looking up: what a fd actually is, what `listen()` reserves, where bytes wait between `write()` and `read()`, and why `accept()` is happy to sleep for hours.

Same calls, different question.
Kurose asked: *how do I build a network application*. Today: *how does the OS make that possible, and what does it cost when several clients show up at once*.

Everything Is a File Descriptor

The UNIX bargain.
Files, pipes, terminals, serial ports, network connections, signals: every kernel-visible resource is reachable through a small integer called a file descriptor.

Why this matters here.
Yesterday your bridge held one fd for the serial port and another for the MQTT socket. Today's TCP server will juggle a fd per client. The same `read`/`write`/`close` works on all of them.

The downside.
Same calls, very different latencies. `read()` from a local file returns now; `read()` from a TCP socket might never return at all.

Pipes, FIFOs, Sockets, Serial Ports

The OS textbook's view (Chapter 6, message passing).
A pipe is two file descriptors: one read end, one write end, both speaking `read`/`write`. A FIFO is a named pipe sitting in the file system. A socket is a fd. Yesterday's serial port is a fd.

The consequence.
One `poll()` call can wait on *all of them at once*. Your gatekeeper on Friday will watch the broker socket, the serial fd to the arm and a pipe from a control thread simultaneously, with one event loop.

Why this matters today.
The hands-on server is TCP-only, but the abstraction has nothing to do with TCP. Replace one fd with a serial port and the same multiplexing code drives the arm.

The BSD Socket API in One Frame

The server life-cycle, four calls.

- `socket()`: create an unbound endpoint, get back a fd.
- `bind()`: attach the endpoint to an address (IP + port).
- `listen()`: mark the fd as accepting connections; kernel allocates a queue.
- `accept()`: pull one ready connection off the queue, get a *new* fd for it.

The client side.
`socket()` then `connect()` runs the TCP handshake and returns when SYN-ACK arrives.

Then it is just bytes.
`read()` and `write()` on the returned fd, exactly like Day 1's serial port.

A Minimal Server, Single Client

```c
int s = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in a = {.sin_family=AF_INET, .sin_port=htons(7777),
                        .sin_addr.s_addr=INADDR_ANY};
bind(s, (struct sockaddr*)&a, sizeof a);
listen(s, 16);                /* queue up to 16 SYNs   */
while (1) {
    int c = accept(s, NULL, NULL);
    handle(c);                /* blocks until client done */
    close(c);
}
```

The shape of the problem.
Client B cannot connect until `handle(c)` returns for A. Two days of MQTT did not show this because the broker dealt with it.

The Listen Queue

What `listen(s, 16)` reserves.
Kernel space for up to 16 half-open connections waiting their turn to be accepted. The number is a backlog hint, not a hard limit.

Two things can fill the queue.
Many clients arriving at once (a burst), or your server taking too long between `accept()` calls (slow service).

When it overflows.
Linux silently drops new SYNs; the client sees a connection timeout, not a clear refusal. A SYN flood looks the same as a slow server to anyone holding a stopwatch.

2 Blocking I/O and Where It Hurts

What Blocking Means

The default.
A socket fd is created *blocking*: `read()` sleeps until at least one byte is available; `write()` sleeps until the kernel's send buffer has room; `accept()` sleeps until a client connects.

Why the kernel offers this.
It is the simplest possible interface. The thread is asleep, costing no CPU, and wakes the instant data arrives.

Why it is a trap with more than one client.
A blocking thread can do exactly one thing at a time. While you are blocked on client A's slow `read`, client B's bytes are stacking up in a buffer you are not looking at.

What Sleeping Actually Costs

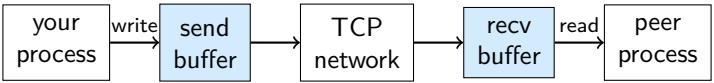
From COMP3322: syscalls and the scheduler.
`read(fd)` traps into the kernel. Empty receive buffer plus a blocking fd: the kernel moves your process from the *ready* queue onto the socket's *wait* queue.

Then nothing happens, for free.
A process on a wait queue costs no CPU. The scheduler does not pick it; it is not even checked. Other ready processes run.

Until an interrupt arrives.
Bytes land at the network card, which raises a hardware interrupt. The kernel routes them into the socket's receive buffer, then walks its wait queue and moves your process back to ready.

Kernel Send and Receive Buffers

Every TCP socket has two queues inside the kernel.
A receive buffer where arriving bytes wait for your `read()`, and a send buffer where your `write()` bytes wait to go out.



The asymmetry that bites.
Buffers are finite. A full *send* buffer means your next `write()` blocks; a full *receive* buffer means TCP itself slows the peer down. The kernel never silently loses your data; it makes you wait.

Bounded Buffers and Backpressure

The OS textbook again.
Zero capacity: sender waits for receiver. Bounded: sender waits if full. Unbounded: never waits, but unbounded queues are a fantasy.

The send buffer is bounded.
When the publisher outpaces the reader, the receive buffer fills. TCP's window shrinks; the sender's `write()` eventually blocks.

The reading.
TCP does not silently drop or grow unbounded. It applies backpressure: slow the sender to the receiver's pace. Sane for streaming, dangerous for an actuator that needs the *latest* command, not the oldest.

Head-of-Line Blocking

The disease today's lab gives you.
Client A's connection is slow; your server's thread is parked inside its `read()` on A. Clients B and C are connected, with bytes ready, but they cannot be served.

Why TCP cannot fix it.
TCP guarantees ordered delivery *within one connection*. Across connections it has no opinion. The blocked thread is the problem, not the protocol.

What the symptom looks like.
Yesterday's per-QoS table looked clean. Today, with one slow laptop on the LAN, the e-stop's tail latency jumps by a full second. The mean does not move; the worst case explodes.

In-Class Quiz: Where Did the Bytes Go?

Your blocking server is parked inside `read(fd_A)` waiting on client A. Client B sends 200 bytes and disconnects cleanly. What happens to B's bytes?

1. Lost: the server never `read`s them.
2. Sit in the kernel's receive buffer for fd_B until someone reads them.
3. Bounced back to B with a TCP RST.
4. Forwarded to fd_A's reader automatically.

In-Class Quiz: Answer

Answer. 2

Why.
TCP and the kernel did their job: the bytes arrived intact and were placed in fd_B's receive buffer. They wait there indefinitely. The buffer is bounded, though; if the server stays blocked, eventually the window closes and B feels back-pressure.

The reading.
The data is not lost, but no one is paying attention to it. The server is awake on the wrong fd. The cure is to wait on *many* fds at once.

3 Multiplexing: Waiting on Many fds

The Three Wrong Answers

One thread per client.
Easy, scales badly. Day 4 shows what a shared serial port does to a multi-thread design; today the cost is context switches and kernel memory per connection.

One process per client.
Even more isolation, even higher cost. The classic Apache prefork model; obsolete for our scale.

Polling in a tight loop.
Non-blocking `read` in a busy-wait: a 100% CPU loop that the OS already knows how to avoid.

The right answer.
Stay in one thread, ask the kernel to watch many fds for you, sleep until *any of them* is ready.

Non-Blocking Mode

The flag.
`fcntl(fd, F_SETFL, O_NONBLOCK)`: now a `read()` with nothing pending returns `-1` with `errno = EAGAIN` instead of sleeping. `write()` acts the same way when the send buffer is full.

What it buys you.
The fd never traps your thread. You can attempt to read, accept a "nothing now" answer, and move on to other fds.

What it does not give you.
Notification: you would still have to ask each fd in turn, burning CPU. Non-blocking by itself is not the solution; it is the precondition for one.

select() and the fd Set

The call.
`select(nfds, &readfds, &writefds, &exceptfds, &timeout)`: hand the kernel three bitmaps of fds you care about, sleep until any is ready, and read the bitmaps back to learn which ones.

Where it shows its age.
Bitmaps are sized by `FD_SETSIZE` (usually 1024); the kernel scans the entire set each call; you rebuild the bitmaps every iteration because `select()` overwrites them.

Why we still mention it.
Every textbook from the 1980s uses `select()`, and every modern alternative is a generalisation of it. Understand this one, and `poll`/`epoll` read as fixes for its specific scars.

poll() and pollfd Arrays

The call.
`poll(&fds[0], nfds, timeout)` where each `pollfd` carries a fd, the events you care about, and a field the kernel fills in.

What it fixes.
No 1024-fd ceiling, no bitmap rebuild between calls, and the API names events (`POLLIN`, `POLLOUT`, `POLLHUP`) instead of three opaque sets.

What it does not fix.
The kernel still walks the whole array each call; you still walk the whole array on return looking for ready fds. Fine for tens of clients, painful at thousands. Day 5 introduces `epoll` for that case.

A poll() Server in C

```c
struct pollfd fds[MAX];
int n = 1;
fds[0].fd = listen_fd; fds[0].events = POLLIN;
for (;;) {
    poll(fds, n, -1);
    if (fds[0].revents & POLLIN) {
        int c = accept(listen_fd, NULL, NULL);
        fds[n++] = (struct pollfd){.fd=c, .events=POLLIN};
    }
    for (int i = 1; i < n; i++)
        if (fds[i].revents & POLLIN)
            handle_one_chunk(fds[i].fd);
}
```

A poll() Server in C (Cont'd)

Reading the loop.
Slot 0 is the listening socket; new connections appear there. Slots 1..n-1 are clients; `POLLIN` on any of them means there is at least one byte to read without blocking.

The big change from the blocking server.
`handle_one_chunk` reads only what is ready and returns immediately. No single client can park the thread, because the thread parks on *the kernel*, not on a fd.

The smaller change.
You no longer call `handle(c)` as a function; you treat a client as a tiny state machine that advances by one chunk per loop pass. Day 5's event loop generalises exactly this shape.

4 Bus, Not Socket

What the Socket Cannot Express

One socket, two addressed ends.
By the time `accept()` returns, the socket is glued to exactly one client. A TCP socket is a 1-to-1 byte pipe with no concept of *topic* or *whoever is interested*.

What the robot system actually wants.
`arm/state` read by every dashboard at once. `arm/estop` from any tool, delivered to the bridge *and* logged. Late subscribers seeing the last retained pose immediately on connect.

The semantic gap.
None of those are socket primitives. They are bus primitives: addressing by topic, fan-out, retention, last will. The kernel will not invent them for you.

A Concrete Topic Graph

What yesterday's pub-sub buys you in one table.

Topic	Publishers	Subscribers	Note
`arm/cmd`	operator, planner	bridge	N-to-1 fan-in
`arm/estop`	any tool, button	bridge, logger	1-to-N fan-out
`arm/state`	bridge	all dashboards	retained
`arm/ack`	bridge	publisher of `cmd`	correlated by SEQ

What a raw TCP socket cannot express.

N-to-1 fan-in requires N sockets and your code to merge them. 1-to-N fan-out requires N sockets and a publisher that knows every subscriber. Retention requires the bridge to remember its last message and replay it. Correlation by SEQ requires you to invent SEQ.

The HCR platform did all of this once, at the broker.

Rebuilding Yesterday from Sockets

Imagine no Mosquitto.
To match yesterday's behaviour with raw sockets, your bridge would have to keep an address book of every subscriber, fan out each publish in a loop, track per-subscriber failures, replay a last-known state on reconnect, and detect dead clients.

This is half of what MQTT brokers are.
The other half is the QoS state machine from yesterday. You would be writing your own broker, badly, in a language designed for files.

The platform lesson, in one line.
The HCR handbook puts it this way: *bus, not socket*. The socket is the right thing under the broker, and the wrong thing for the robot fleet to speak directly.

Where Yesterday's Story Fits

Sockets are the floor.
Every MQTT connection on the lab LAN is a TCP socket between a client and Mosquitto. Today's `poll` server is what the inside of a tiny broker looks like.

QoS lives one layer up.
Yesterday's at-most-once / at-least-once / exactly-once contracts ride on top of these byte pipes. The broker uses sockets to build a bus.

Friday will close the loop.
The capstone's gatekeeper is a small program that owns the serial socket (the actuator) and faces the bus (MQTT). Today you are writing a sketch of its lower half.

Hotaru: the Named Access Point

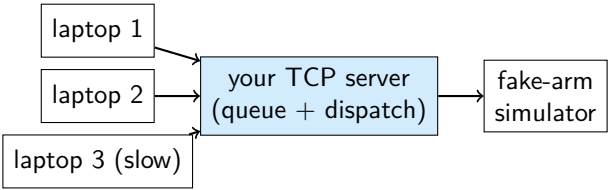
Why give the bus a name.
Once the broker offers topics, fan-out and retention, your code stops thinking about sockets. It thinks about one object that speaks publish, subscribe, last-will. The HCR platform calls it *Hotaru*.

Transport swap as a non-event.
Hotaru makes the path to the arm a configuration choice. The caller publishes to `arm/cmd`; Hotaru routes that over MQTT-TCP, an in-process broker, or a serial fd to a real arm.

In our C lab.
One `struct` of function pointers: `publish`, `subscribe`, `disconnect`. Friday's gatekeeper holds one such struct; we swap the implementation, not the call sites.
The abstraction is the point, not the wire.

5 Project Hands-on Preview: Build the Server

The Hands-on Chain, Today



One simulator, many command sources.
Each laptop opens a TCP connection and streams Day 1 frames; the server forwards valid frames to the single serial fd.

We supply the noisy laptop.
One of the supplied clients reads slowly on purpose. Your job is to feel its effect, then make it stop mattering.

Hands-on Tasks

Task 1: the blocking baseline.
Build a single-threaded `accept`-then-`handle` server. Measure end-to-end latency from publish to ACK with one fast and one slow client. Plot the worst case, not just the mean.

Task 2: switch to `poll()`.
Rewrite around the `poll` loop above. Keep the same workload; replot. Two numbers should change dramatically.

Task 3: pin the cause.
Replace the slow client with the supplied flood client (fast, but high rate). Where does latency reappear? Is it the kernel buffer, the serial link from Day 1, or your own code?

Hands-on Tasks (Cont'd)

Pass condition.
A clean before/after plot showing that a slow client no longer affects fast clients' tail latency once `poll()` is in.

Keep what you build.
Tomorrow's locking lab uses this multi-client server as its testbed; Friday's event-loop rewrite replaces `poll()` with `epoll()` on the same skeleton; the capstone gatekeeper inherits this shape.

Day 3 Summary

Sockets are files, with a worse worst case.
Same `read`/`write` as a serial port, but a TCP `read()` can sleep forever, and a blocked thread serves nobody.

Multiplexing is the cure for one slow client.
`select`, `poll`, `epoll`: all solve the same problem, wait on many fds, do not wait on *a* fd.

Sockets are not a bus.
Fan-out, topic addressing and retention are not in the kernel. The broker built them on top, and that is why yesterday felt easier than today's socket work.

Tomorrow.
Two threads share the single serial port. What goes wrong, and which lock fixes it without inviting deadlock?

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template